SketchMind: Toward Executable Pen-Based Sketching for Constructing Sorting Algorithms

Yang Wu
yang.wu@inf.ethz.ch
ETH Zurich
Zurich, Switzerland

Clément Pit-Claudel
clement.pit-claudel@epfl.ch
EPFL
Lausanne, Switzerland

April Yi Wang
april.wang@inf.ethz.ch
ETH Zurich
Zurich, Switzerland

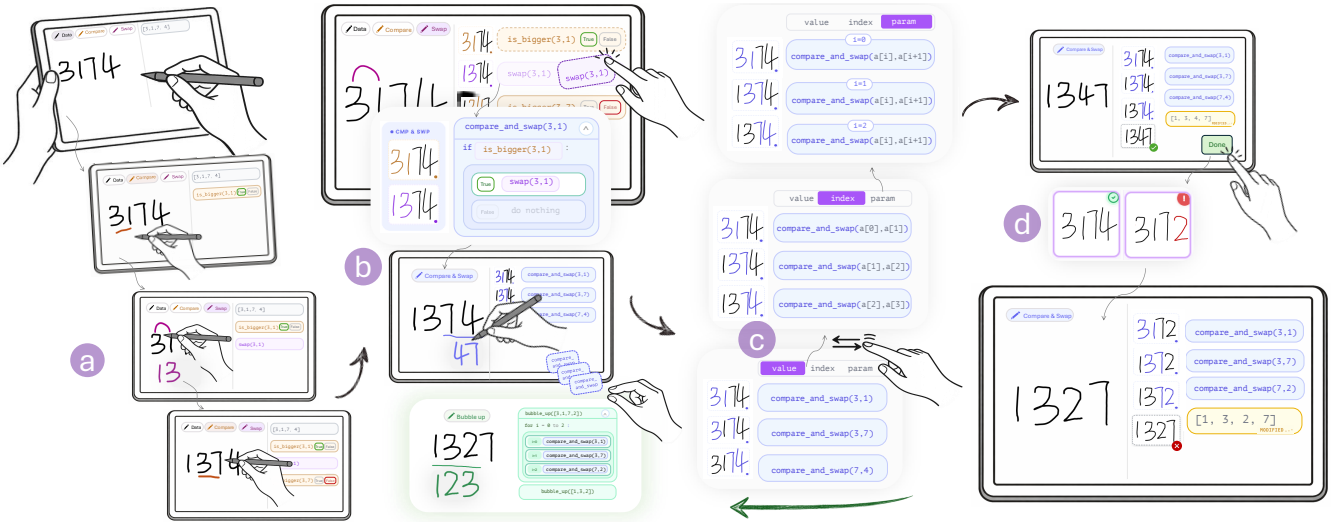


Figure 1: SketchMind usage example with four core features: (a) Executable Tracing: a learner sketches an input array and performs primitive operations (Data, Compare, Swap), which are immediately executed and recorded as trace cards; (b) Reusable Pens: the learner merges trace cards (e.g., dragging swap into the true branch of is_bigger) to create a reusable Compare&Swap pen; (c) Multi-level trace views: the learner inspects the same procedure across value, index, and parameter views; (d) Cross-Instance Validation: after the learner clicks Done, if a hidden flaw remains, the system generates a minimal modified input on which the same logic ends unsorted, revealing the missing step for refinement.

Abstract

Pen-and-paper tracing is commonly used to think through algorithms before writing code, allowing learners to step through concrete executions and explore possible solutions. However, this process does not readily support abstraction. As traces grow, they lose structure. Steps accumulate into long sequences and are often skipped or compressed, leading to incomplete or inconsistent reasoning. As a result, learners struggle to extract reusable algorithmic patterns or generalize beyond individual instances. We present SketchMind, a proof-of-concept sketch-based programming system in which users construct executable traces of sorting algorithms through pen-based interactions. By performing operations directly on visual representations (e.g., arrays and variables) using pen strokes, users can construct and refine sorting algorithms without writing code. SketchMind supports four key capabilities: (1) executable tracing, where actions are recorded and replayed as runnable steps; (2) reusable pens, which encapsulate trace segments into user-defined procedures; (3) multi-level trace views across values, indices, and symbolic parameters; and (4) cross-instance validation, which applies learned procedures to new inputs and generates minimal counterexamples.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; • **Applied computing** → **Interactive learning environments**.

Keywords

Algorithmic reasoning, Sketch-based interaction, Executable tracing, Programming by demonstration;

ACM Reference Format:
Yang Wu, Clément Pit-Claudel, and April Yi Wang. 2026. SketchMind: Toward Executable Pen-Based Sketching for Constructing Sorting Algorithms.

UIST '26, November 02–05, 2026, Detroit, MI, USA

ACM ISBN 979-8-4007-2856-3/2026/11
https://doi.org/10.1145/3830398.3830500

In *The 39th Annual ACM Symposium on User Interface Software and Technology (UIST '26), November 02–05, 2026, Detroit, MI, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3830398.3830500

1 Introduction

When learning algorithms, learners often begin by stepping through concrete examples and mentally simulating how values change over time before writing code. This process, commonly performed with pen and paper, externalizes learners' mental execution and supports early reasoning about algorithm behavior prior to implementation [1, 8, 9]. However, while tracing exposes what happens during execution, it does not readily support constructing why it works as a generalizable procedure.

As traces grow, they become increasingly difficult to manage and reason about. Steps accumulate into long sequences that are tedious to maintain and easy to corrupt, increasing the cognitive load placed on learners [42]. For example, when tracing bubble sort on an array, learners must repeatedly write out intermediate states after each comparison and swap, quickly resulting in lengthy and hard-to-follow traces. While tracing exposes individual execution steps, it provides little support for organizing these steps into structures that allow learners to identify recurring patterns and connect concrete operations to higher-level abstractions [48]. Learners may repeatedly perform compare-and-swap operations without recognizing how these steps form a recurring subgoal within the inner loop of bubble sort [5]. More fundamentally, tracing remains tied to a single example, and learners may form explanations that fail to generalize across inputs or edge cases, echoing the generalization challenge in programming by demonstration [20, 21]. For instance, tracing bubble sort on a nearly sorted array may lead to the incorrect conclusion that only a few swaps are needed.

Beyond traditional pen-and-paper tracing, existing tools for learning algorithms tend to separate tracing from construction. Tracing-based visualization tools (e.g., [13, 23]) support understanding how existing programs execute, but treat traces as fixed artifacts of a predefined algorithm, offering little support for designing algorithms from scratch. Annotation tools (e.g., [1]) digitize traces to improve editability, but remain non-executable and do not fundamentally address the core challenges of managing, structuring, and validating execution traces in pen-and-paper tracing. Together, these approaches leave a critical gap: they do not support constructing algorithms directly from traces through sketch-based interaction, pushing learners back to manual pen-and-paper workflows.

To address these challenges, we introduce SketchMind, a sketch-based system that explores how users might construct algorithms directly from execution traces through direct manipulation. Rather than treating traces as endpoints, SketchMind supports a bottom-up workflow in which users derive structured procedures by acting on concrete executions. SketchMind provides four key capabilities: executable tracing, structural composition, multi-level trace views, and minimal counterexample generation. Users work through an example by executing primitive operations with pen strokes, which SketchMind records as an executable trace. The system allows learners to progressively compose related execution steps into reusable units to form larger algorithmic structures. SketchMind further supports abstraction across different levels, allowing learners to move from procedures defined over concrete values to reasoning over array indices and eventually symbolic parameters, making recurring execution patterns explicit and reusable. To support reasoning beyond a single example, SketchMind allows users to replay constructed procedures on new inputs and automatically generates counterexamples by minimally modifying the original example while preserving the learner's execution path. While the current implementation focuses on a small set of algorithms (e.g., Bubble Sort and Pancake Sort), we position SketchMind as an initial step toward more general systems that support constructing a wider range of algorithms from execution.

We conducted a preliminary study with 15 students and 5 instructors ($N = 20$) to explore how SketchMind supports algorithm construction from tracing data. Results indicate strong perceived value of counterexample-based validation, while reusable pens and progressive abstraction help learners move from concrete trace steps to structured, generalizable procedures. From a pedagogical perspective, instructors considered SketchMind well suited for introductory learning, supporting in-class demonstration and self-paced homework.

This paper makes the following contributions:

- We explore how users can build algorithms by working directly with execution traces through sketch interaction, which we refer to as *executable sketches*.
- SketchMind, a proof-of-concept system that instantiates executable sketches through two concrete algorithms and supports users in managing, structuring, and validating execution traces, along with preliminary insights from a user study.
- A counterexample-based approach for revealing gaps in constructed procedures. SketchMind generates minimal variations of the input that expose when a procedure does not generalize, allowing users to identify missing steps. This is implemented using a constraint-based method with a Satisfiability Modulo Theories (SMT) solver to efficiently find nearby failing cases.

2 Related Work

2.1 Pen-and-Paper Tracing in Programming Education.

Tracing has long been recognized as a foundational practice in introductory programming, especially for building a concrete mental model of program state changes. A large multi-national study showed that many CS1 learners struggle with reading and tracing even short code fragments, suggesting that tracing skills are a prerequisite for higher-level problem solving rather than a by-product of it [27]. Moreover, novices often struggle with the *cognitive load* of simultaneous state tracking and logic induction [42]. Subsequent work further showed that novices often focus on local statements instead of relational program structure, whereas experts more frequently reason at a structural level [28].

To mitigate this, SketchMind functions as a form of cognitive scaffolding [50], externalizing working memory by automating state updates. SketchMind bridges this gap by allowing learners to construct executable logic *bottom-up*. This shifts tracing from post-hoc program interpretation to incremental construction.

2.2 Algorithm Visualization and Execution Tracing Tools

Algorithm visualization (AV) has evolved from early static systems like BALSA [3] to highly interactive environments. Prior empirical studies report that visualization is most effective when learners actively engage with it (e.g., prediction, interaction, or question answering), rather than passively watching animations [11, 17, 34]. To this end, systems like JHAV'E and JHAV'E-II integrated pseudocode, snapshots, and interactive questioning to bolster engagement [35]. Execution tracing tools further support learning by allowing users to step through code and observe state changes. However, most existing tracing tools (e.g., Python Tutor [13]) require learners to possess programming knowledge or have a predefined code snippet to trace. This creates a significant barrier for novices who struggle with syntax while trying to grasp high-level algorithmic logic.

In contrast to traditional AV and tracing tools centered on pre-authored content or code-dependent execution, SketchMind emphasizes learner-authored execution logic. By enabling learners to "sketch" their own tracing rules without writing formal code, SketchMind aligns with live programming principles [43]. This tight feedback loop between sketching (editing) and observing (execution) allows learners to immediately see the logical consequences of their mental models, fostering a deeper understanding of how abstract algorithmic logic is realized through concrete execution.

2.3 Programming by Demonstration

Programming by Demonstration (PBD) enables users to construct programs by demonstrating desired behaviors through example actions, from which systems derive reusable procedures [10]. Originating from research in human-computer interaction and intelligent user interfaces, PBD lowers the barrier to programming by allowing users to specify behavior through concrete interactions rather than formal code. A central challenge in PBD is the so-called "data description" problem, i.e., how to lift operations performed on specific objects into abstract representations that generalize across inputs. This requires identifying relevant structure (e.g., positions or indices) while disregarding incidental details. Prior work shows that simple recording-and-replay approaches often produce brittle behaviors, while attempts to generalize from demonstrations can introduce ambiguity due to underspecified user intent [20]. Consequently, research in interactive and end-user programming emphasizes keeping users in the loop, enabling them to inspect, guide, and refine automated behaviors [6, 14, 31].

SketchMind addresses this challenge through progressive abstraction, recording concrete sketch-based operations (e.g., swaps or comparisons) and encapsulating them as reusable *pens*, while providing a UI-driven pathway to lift these actions into indexed or parameterized procedures (e.g., $a[i]$). This allows users to effectively "teach" the system an algorithm through example-based tracing, reducing the barrier to formalizing complex logic.

2.4 Sketching Tools and No-Code Systems

Sketching provides a low-fidelity yet highly expressive interface that closely mirrors the fluidity of human thought during early-stage problem solving [45]. Prior work has leveraged sketch-based interaction to bridge the gap between informal mental models and formal representations across diverse domains. In mathematics education, systems such as MathPad support the recognition of handwritten expressions to construct structured mathematical models, while Noyon [39] allows learners to explore algebraic concepts through sketch-based grounding. Similarly, Augmented Physics [12] transforms static textbook diagrams into interactive simulations, enabling more dynamic reasoning about physical processes. In computer science education, sketch-based systems have been used to support algorithmic thinking and program construction. AlgoSketch [26] enables users to author and execute pseudocode-like descriptions, while systems such as *CSTutor* [4] and *ThinkInk* [18] employ pen-based interfaces to visualize data structures and support reasoning about algorithms. More recent work, such as Code Shaping [52], further explores the integration of free-form sketching with automated code generation. However, these tools primarily treat pen strokes as static visual representations or isolated code triggers, lacking the ability to compose trace operations dynamically. In parallel, no-code environments—ranging from block-based programming (e.g., Scratch [36]) to trigger-action platforms (e.g., IFTTT [44]), lower the barrier to entry by replacing textual syntax with visual, constrained abstractions. However, while these systems reduce syntax errors, they often impose a rigid structure that may prematurely constrain a learner's exploratory tracing process.

SketchMind bridges these paradigms by unifying the fluidity of pen-based sketching with composable execution of structured abstractions. Specifically, SketchMind assigns operational semantics to pen strokes, enabling concrete trace steps to be composed in a manner analogous to higher-order functions (e.g., placing a *swap* operation within a conditional to create a reusable *compare-and-swap* pen). This preserves the immediacy of paper-like sketching while supporting parameterized algorithm construction.

3 Design Motivations

To understand the practical hurdles in algorithm learning, we analyze the traditional transition from manual tracing to formal logic construction through a representative scenario. Consider Alice, a student learning Bubble Sort. She begins by tracing the algorithm on a sheet of paper using an input array $A = [3, 1, 7, 4]$.

Alice starts by writing the initial state. She compares 3 and 1, crosses out the old array, and redraws the new state $[1, 3, 7, 4]$. By the time she reaches the end of the first pass, her paper is cluttered with scratched-out numbers and fragmented sequences. As the complexity grows, the number of manual "renders" required for a full nested-loop execution becomes overwhelming. To save effort, Alice begins skipping intermediate states or compressing multiple swaps into a single line. This tedious manual process is not only time-consuming but prone to clerical errors, such as incorrectly transposing a digit during redrawing.

C1: Manual Bookkeeping and Trace Explosion. As intermediate states build up, tracing becomes hard to manage and discourages complete execution. Redrawing steps is error-prone, and without a way to check the result, mistakes often go unnoticed.

While Alice successfully performs several compare-and-swap operations, these actions remain isolated and tied strictly to the values 3, 1, 7, and 4. When she attempts to mentalize the "inner loop" of the algorithm, she struggles to bridge the gap between physical pen strokes and the abstract logic of a conditional swap. There is no mechanism in her paper sketch to "capture" a successful sequence of moves and reuse it as a functional unit; every new pair of elements requires her to restart the same mental simulation from scratch.

> **C2: The Abstraction Leap.** Learners struggle to transition from concrete value manipulation to structural representations (e.g., loops and conditionals). Traces remain tied to specific instances, providing no support for organizing disjointed steps into reusable algorithmic blocks.

After one pass, Alice's specific example $[1, 3, 4, 7]$ happens to appear sorted. Without an automated way to verify her logic, she prematurely concludes that a single linear pass is sufficient for any input. Because her paper trace is static and non-executable, she cannot easily "replay" her reasoning on a different, more chaotic input (e.g., $[4, 3, 2, 1]$) to discover the necessity of the outer loop. If she makes a logic error early on, there is no system to provide immediate feedback, allowing misconceptions to persist throughout her study session.

> **C3: Instance Overfitting.** Reasoning grounded in a single "happy path" example fails to generalize, with no mechanism to surface counterexamples. While multiple input–output test cases might be available (e.g., provided by instructors or platforms like *LeetCode*), they mainly serve as correctness checks. When a solution fails, these cases do not explain why, and are often too different from the learner's working example to help them revise their reasoning.

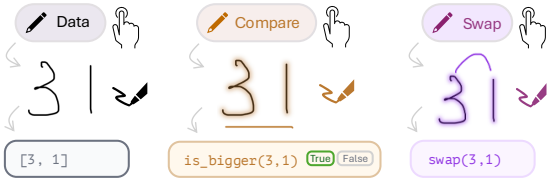
4 System Design

To bridge the gap between concrete tracing and abstract algorithm construction, SketchMind is guided by four design goals:

- **C1 –> DG1: Support executable tracing.** Transform pen-based gestures into executable operations to automate state tracking, reducing the tedious bookkeeping of manual tracing.
- **C2 –> DG2: Support structural organization of traces.** Provide a container mechanism (*"pens"*) to encapsulate disjointed trace steps into reusable logical blocks like loops or conditionals.
- **C2 –> DG3: Support multi-level trace views.** Scaffold the elevation of logic from concrete instance values to positional and generalized parameters (e.g., *Index i*) through a three-level abstraction model, helping learners identify recurring patterns and generalize them into reusable algorithmic structures.
- **C3 –> DG4: Support validation across inputs.** Enable users to replay derived logic on new data inputs and automatically surface counter-examples to ensure algorithmic robustness.

4.1 Executable Tracing

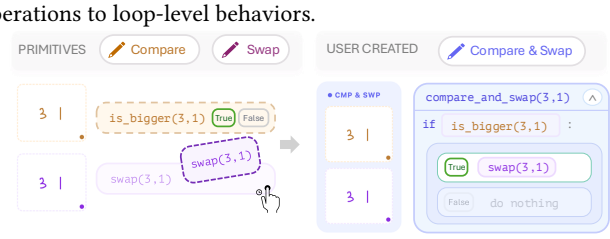
SketchMind supports executable tracing by mapping pen strokes to instructor-defined primitive operations and immediate state updates. These primitives define atomic operations over program state (e.g., data access, comparison, and transformation), supporting intuitive expression of algorithmic behavior.



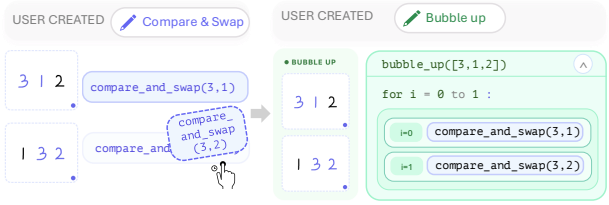
For Bubble Sort, SketchMind includes `Data`, `Compare`, and `Swap`. Learners start by sketching an input with the `Data` pen, after which SketchMind recognizes the digits, instantiates an executable array state, and binds subsequent strokes to typed operations. From there, learners perform algorithmic steps by first selecting an operation pen and then drawing strokes of any shape across the elements to which it should apply. For example, the learner can perform algorithmic steps, such as comparison, by selecting the `Compare` pen and drawing between two elements. During this stroke interaction, the referenced numbers are highlighted with a halo to indicate selection. SketchMind then executes the comparison, highlights the referenced values, and appends a corresponding `is_bigger` compare card to the trace panel (e.g., `is_bigger(3, 1)=true`). To modify element order, learners select the `Swap` pen and draw between two positions. The system animates the exchange, inserts a `swap` card into the trace, and updates the array state for subsequent operations. Each pen stroke is thus compiled into a persistent, typed trace card, and the linked cards (*Data* → *Compare* → *Swap*), yielding an executable trace that can be replayed, inspected, and revised during algorithm construction. More generally, different algorithms define distinct primitive operations that capture their core computational logic. For example, in Pancake Sort, primitives include `find_max` and `flip`. In SketchMind, a range selection stroke denotes `find_max`, identifying the maximum element within the selected region, while a range stroke also denotes `flip`, reversing the selected subsequence.

4.2 Reusable Pens

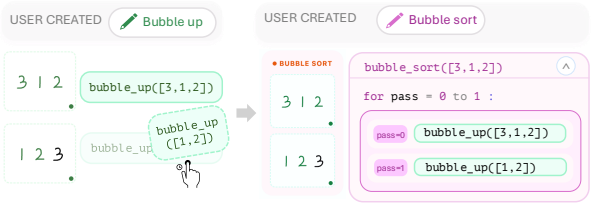
To support algorithmic structuring, SketchMind enables learners to compose primitives into reusable logic blocks through conditional and iterative composition. Trace fragments can be merged into *user-created pens*, allowing hierarchical construction from if-else operations to loop-level behaviors.



For `if-else` composition, learners combine a predicate with its corresponding action. For example, `is_bigger` and `swap` can be merged into a `compare_and_swap` block, where the swap is executed only when the condition holds. This block is promoted to a reusable `Compare & Swap` pen that can be applied directly to any pair of elements while preserving the underlying logic.

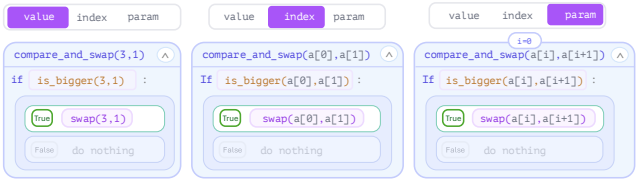


For `for`-style iteration, the repeated application of a reusable block over a specified range forms higher-level operations. For instance, applying a `compare_and_swap` sequence across adjacent elements yields a `Bubble up` pen, which encapsulates a single left-to-right pass to move the maximum element to the end of the range. Subsequent composition of these `bubble_up` passes produces a `Bubble Sort` pen that captures nested-loop behavior, progressively shortening the unsorted suffix until the array is ordered.



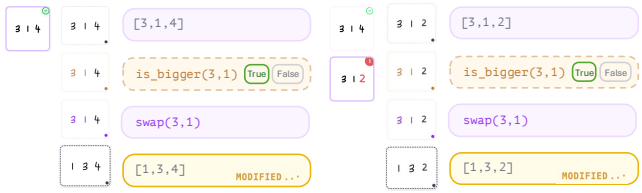
More generally, these compositions adapt to the diverse structural requirements of different algorithms. In Pancake Sort, for example, SketchMind allows learners to couple a `find-max` primitive with a `flip` operation to form a higher-level `flip_to_front` block. This composition captures a specific data-driven dependency, where the index identified by the maximum search directly determines the range of the subsequent reversal.

4.3 Multi-level Trace Views



SketchMind supports abstraction through three coordinated views of the same procedure (illustrated with `compare_and_swap`). At the *value view*, learners see concrete operands from the current trace instance (e.g., comparing 3 and 1), which helps them verify immediate execution outcomes. At the *index view*, the same logic is lifted to positions (e.g., $a[0]$ and $a[1]$), making adjacency and update locations explicit. At the *parameter view*, the procedure is expressed symbolically (e.g., $i = 0$), allowing the same operation to be reused across different array segments and contexts. This progression lets learners observe how one concrete step becomes a reusable rule: values explain *what happened*, indices explain *where it happened*, and parameters explain *how it generalizes*. When the index variable i changes during iteration, learners can directly observe the compared pair shifting along the array $(a[i], a[i+1])$, making the loop's traversal pattern and boundary behavior explicit across executions.

4.4 Counterexample Generation



To support validation across inputs, SketchMind keeps the user trace unchanged and searches for the nearest failing input. Specifically, the solver preserves the constructed control flow and identifies the minimal input perturbation for which the replay produces an incorrect result (e.g., an unsorted array). For instance, if a learner builds logic on 3, 1, 4 that only compares and swaps the first two elements, SketchMind can generate a nearby counterexample such as 3, 1, 2, where replaying the same pen-trace logic still leaves the array unsorted. This feedback does not auto-repair the solution; instead, it helps learners identify the missing steps and complete a full Bubble Sort procedure that generalizes across inputs.

4.5 System Walkthrough

We illustrate the core interaction of SketchMind through a partial Bubble Sort trace on $A = [3, 1, 7, 4]$. We follow Alice again as she incrementally constructs and refines a solution through iteration.

4.5.1 Step 1: Creating an initial trace. Alice begins by the `Data` pen and writes the input values `3, 1, 7, 4`. SketchMind recognizes the handwritten digits and instantiates an executable array state. Alice then selects the `Compare` pen and draws between 3 and 1; SketchMind highlights the selected values with a halo, executes the predicate, and appends an `is_bigger(3,1)` card to the trace. Next, Alice selects the `Swap` pen and draws between the same pair; SketchMind animates the exchange, updates the array state, and records a `swap(3,1)` card. Alice repeats this interaction for (7, 4), resulting in `[1, 3, 4, 7]`. At this stage, she has constructed a partial execution trace corresponding to local sorting behavior, without explicitly encoding a full algorithm.

4.5.2 Step 2: Structuring behavior into reusable logic. Noticing that similar compare and swap operations recur, Alice groups multiple trace steps into a reusable unit. SketchMind allows her to encapsulate these steps into a user-defined pen, forming a higher-level operation that can be applied without re-sketching individual actions. To do so, Alice drags the `swap` card into the `true` branch of `is_bigger`. This creates a merged `compare_and_swap` block with explicit conditional semantics: if `is_bigger` is true, execute `swap`; otherwise, do nothing. This interaction does not merely group steps, but constructs explicit conditional semantics: the `swap` operation

becomes guarded by the predicate `is_bigger`, forming a conditional execution block. SketchMind then promotes this merged block to a reusable user-defined pen (`Compare&Swap`), allowing the user to invoke the same logic directly on other adjacent pairs.

4.5.3 Step 3: Abstracting from concrete values to generalized rules. Initially, Alice's procedure is tied to specific values (e.g., `(3, 1)`). She then lifts this representation to operate over indices (e.g., `(a[0], a[1])`), and further to symbolic parameters (e.g., `(a[i], a[i+1]), i = 0`). Through this progression, Alice transitions from instance-specific reasoning to a parameterized rule that can be reused across positions and inputs. SketchMind makes this transformation explicit by maintaining aligned views across value, index, and parameter representations. This enables Alice to reason about both local behavior and reusable loop patterns.

4.5.4 Step 4: Revealing incompleteness through counterexamples. Although Alice's constructed procedure correctly transforms the array `[3, 1, 7, 4]` into a sorted state `[1, 3, 4, 7]`, her reasoning remains incomplete. After performing a single pass of adjacent comparisons, Alice observes that the array appears sorted and prematurely concludes that no further steps are needed. As a result, her procedure encodes only a single iteration of local compare-and-swap operations, rather than a complete sorting algorithm.

When Alice invokes validation, SketchMind generates a minimal counterexample that exposes this flaw. Specifically, the system produces a nearby input by minimally modifying the original array—for example, replacing (4) with (2) to obtain `[3, 1, 7, 2]`. Replaying Alice's procedure on this input reveals that a single pass is insufficient: the array remains unsorted after execution.

This counterexample highlights that Alice's current logic overfits to the original instance and fails to generalize. Rather than prescribing a fix, SketchMind surfaces the precise condition under which the procedure breaks down, prompting Alice to refine her approach by introducing additional passes.

4.5.5 Iterative Refinement. To address the counterexample, Alice reorganizes her trace into a reusable procedure that captures a single left-to-right pass of adjacent compare-and-swap operations, which she defines as a `Bubble up` pen. This pen encapsulates the behavior of moving the largest unsorted element to its correct position at the end of the array.

Alice then applies the `Bubble up` pen iteratively. By repeating this operation for three passes on the array of length four, she progressively reduces the unsorted portion and eventually obtains a fully sorted array. Through this process, Alice transitions from a single-pass, instance-specific trace to a structured, iterative algorithm that generalizes across inputs.

4.6 Implementation

4.6.1 System Architecture. SketchMind processes user input via three layers, as shown in Appendix Figure 4. The *sketch layer* handles two pen types and touch gestures: a digit pen recognizes handwritten numerals via Google Gemini 3.0 Flash and binds them to array slots; an operation pen, the selected pen specifies the operation, while the stroke path identifies the elements involved. For example, with the Swap pen selected, learners can draw any-shaped stroke across two numbers to swap them, as the system relies on the path's intersection with their bounding boxes rather than the stroke's shape. Touch gestures restructure the card topology via merge and split. The *execution engine* maintains a versioned step sequence; inserting or deleting a step triggers downstream state recomputation and replays both array animations and original ink across all instances via multi-instance replay. The *symbolic execution engine* compiles the operation sequence into formula, checks its correctness, and finds the closest failing input as a counterexample, which is injected as a new instance and fed back into the execution engine for further practice.

Table 1: Abstract trace and execution derivation. Original/Modified: initial input vs. counterexample. CF (control flow) denotes guard evaluations, i.e., true (✓) or false (×).

Abstract	Original	CF	Modified	CF
`if a[0] < a[1]`	`3 > 1`	✓	`3 > 1`	✓
`then swap`	→ swap		→ swap	
`if a[1] < a[2]`	`3 < 7`	×	`3 < 7`	×
`then swap`	→ no swap		→ no swap	
`if a[2] < a[3]`	`7 > 4`	✓	`7 > 2`	✓
`then swap`	→ swap		→ swap	
`if a[0] < a[1]`	(not executed)	–	`1 < 3`	×
`then swap`	–		→ no swap	
`if a[1] < a[2]`	(not executed)	–	`3 < 2`	✓
`then swap`	–		→ swap	
`if a[2] < a[3]`	(not executed)	–	`3 < 7`	×
`then swap`	–		→ no swap	

4.6.2 SMT-based Counterexample Generation. In Section 4.5.4, Alice traces Bubble Sort on the input `[3,1,7,4]` and performs a single left-to-right pass of adjacent compare-and-swap operations. On this input, the resulting array `[1,3,4,7]` appears sorted, so Alice may incorrectly conclude that no further steps are needed. The key question is whether this learned trace generalizes beyond the original example. Rather than asking Alice to solve a different problem, SketchMind searches for a nearby input that follows the same execution steps but reveals a failure. For example, as shown in Table 1, replacing 4 with 2 gives `[3,1,7,2]`. The first pass has the same control flow, performing the same comparisons and swaps as before, preserving the original execution. However, this new input requires additional passes to be sorted, exposing the missing step.

Formally, given an input array $A = (a_0, \ldots, a_{n-1})$, Alice sketches a trace T, where each step is encoded as a symbolic transition constraint. Each trace operation is translated into an algorithm-specific transition macro. For Bubble Sort, a comparison introduces a Boolean guard (e.g., $a_i > a_j$) over two array positions, a swap exchanges their values, and a conditional swap updates the state only when the guard holds. Higher-level operations, such as a Bubble Sort pass, are expanded into sequences of these primitive transitions. Supporting another algorithm therefore requires defining the corresponding operation macros; for example, Pancake Sort requires macros for selecting the maximum element in a range and reversing a prefix. Part 2 of Table 2 summarizes these algorithm-specific

Table 2: Formal semantics of SketchMind.

Component	Formal Constraints
Part 1: General Interactions and State	
Domain and state	$A^{(0)} \in [1,99]^n \cap \mathbb{Z}^n, \quad A^{(t)} = (a_0^{(t)}, \ldots, a_{n-1}^{(t)})$. $A^{(t)}$ is its array state after replay step t.
LCS equality predicate	$\mathrm{Eq}(u_p, r_q) \leftrightarrow (\tau_p = \sigma_q) \wedge (x_p = \alpha_q) \wedge (y_p = \beta_q)$.
LCS dynamic programming	$L_{0,q} = L_{p,0} = 0$, $L_{p,q} = \mathrm{ite}(\mathrm{Eq}(u_p, r_q), L_{p-1,q-1} + 1, \max(L_{p-1,q}, L_{p,q-1}))$.
Part 2: Algorithm-Specific Macros	
Compare on (i,j) at step t	$g_t \leftrightarrow (a_i^{(t)} > a_j^{(t)}), \qquad A^{(t+1)} = A^{(t)}$.
Bare swap on (i,j) at step t	$a_i^{(t+1)} = a_j^{(t)},\ a_j^{(t+1)} = a_i^{(t)},\ \forall k \notin \{i,j\} : a_k^{(t+1)} = a_k^{(t)}$.
Conditional swap on (i,j) at step t	$a_i^{(t+1)} = \mathrm{ite}(a_i^{(t)} > a_j^{(t)}, a_j^{(t)}, a_i^{(t)}),\ a_j^{(t+1)} = \mathrm{ite}(a_i^{(t)} > a_j^{(t)}, a_i^{(t)}, a_j^{(t)}),\ \forall k \notin \{i,j\} : a_k^{(t+1)} = a_k^{(t)}$.
Bubble-sort pass on $[l,r]$	$\Phi_{\text{bubble\_pass}} \equiv \bigwedge_{i=l}^{r-1} \Phi_{\text{cond\_swap}(i,i+1)}$.
Bubble-sort algorithm	$\Phi_{\text{bubble\_sort}} \equiv \bigwedge_{p=0}^{n-2} \bigwedge_{i=0}^{n-2-p} \Phi_{\text{cond\_swap}(i,i+1)}$.
Part 3: General Feedback and Verification	
Sortedness / failure	$\mathrm{Sorted}(A') \equiv \bigwedge_{k=0}^{n-2}(a'_k \le a'_{k+1})$, $\mathrm{Fail}(A') \equiv \neg\mathrm{Sorted}(A')$.
Path classification	$\mathrm{Incomplete} \leftrightarrow (\exists q : \neg\mathrm{matched}_q) \vee (\exists t : \tau_t = \mathrm{bare\_swap})$, $\mathrm{Success} \leftrightarrow \neg\mathrm{Incomplete}$.
SMT objective (minimize distance)	$\min \sum_{k=0}^{n-1} \lvert a'_k - a_k \rvert$.
SMT constraints (incomplete path)	$\Phi_{\text{replay}}(A) \wedge \mathrm{Fail}(A')$.
SMT constraints (success path)	$\Phi_{\text{replay}}(A) \wedge \mathrm{Fail}(A') \wedge \bigwedge_{\mathbf{b} \in \mathcal{H}} \bigvee_{k=0}^{n-1}(a_k \neq b_k)$. $\mathcal{H}$: previously generated counterexamples.

semantics. Before generating a counterexample, SketchMind uses the Longest Common Subsequence (LCS) predicate to compare the learner's operation sequence with the reference algorithm trace. Operations are matched by their type and operand positions. Unmatched reference operations indicate that the learner's trace omits steps required by the algorithm and is therefore structurally incomplete. The SMT solver then searches for a minimally modified input A' such that replaying the same trace T leads to an unsorted result:

$$\min_{A'} \sum_{k=0}^{n-1} |a'_k - a_k|$$

subject to

$$Replay(T, A') \ \wedge \ \exists i \in [0, n-2],\ a'_i > a'_{i+1}.$$

The objective keeps the counterexample close to the learner's original input, while the failure constraint requires the replayed trace to leave the final array unsorted. If such an A' exists, it serves as a counterexample (e.g., $A' = [3,1,7,2]$) showing that the trace does not generalize.

5 Preliminary User Study

We conducted a preliminary user study to examine SketchMind's usability and its potential to support algorithm construction from both learner and instructor perspectives. We recruited 20 participants, including 15 students and 5 instructors, most of whom are affiliated with the first author's university. Participants are identified using IDs prefixed with "P" (students) and "E" (instructors). As shown in Appendix Table 3, participants represented a diverse set of fields (e.g., computer science, engineering, data science, and linguistics) and academic levels from undergraduate to PhD. The instructors have teaching experience across secondary-school computer science, youth programming workshops, and university-level CS courses. All participants had prior experience with Bubble Sort sorting algorithm, which was used in the familiarization task, but were not familiar with Pancake Sort, which was used to assess interaction with a novel problem. The study was approved by the institutional ethics review board, and participants received CHF 25 as compensation for their time.

5.1 Study Protocol

We designed a three-stage protocol to examine how participants interact with SketchMind across familiar and novel algorithmic tasks. The study was conducted in person using an 11-inch iPad Pro and an Apple Pencil. Each session was scheduled for 60 minutes, though actual completion time varied, with some participants finishing earlier.

5.1.1 Stage 1: Familiarization via Bubble Sort (25 min). We selected *Bubble Sort* as a representative familiar algorithm to allow participants to acclimate to the system's interface without intrinsic cognitive load from new algorithmic logic. We first presented a brief instructional video to help participants recall the algorithm, after which they freely sketched how it worked using pen and paper, based on their own understanding. This allows us to capture their initial representations and reasoning processes. Participants then completed four structured tasks in SketchMind, each progressively introducing a core affordance: executable sketching, reusable pens, abstraction, and counterexample-based validation. After each task, they provided Likert-scale ratings and qualitative justifications.

5.1.2 Stage 2: Exploration via Pancake Sort (25 min). To examine how participants engage with SketchMind on an unfamiliar problem, we introduced *Pancake Sort*[1], an algorithm unfamiliar to all participants, even the instructors. Similar to Stage 1, participants first attempted a paper-and-pencil sketch to identify initial conceptual gaps. Participants then used SketchMind to interactively construct

[1]Pancake Sort is a sorting algorithm that sorts an array by repeatedly flipping prefixes, similar to flipping a stack of pancakes.

and test their own sorting strategies. We employed a "think-aloud" protocol to observe exploration behaviors and breakdowns. We recorded the types of assistance required, distinguishing between *algorithmic hints* (logic-related) and *interface hints* (tool-related). Finally, we conducted a brief post-task assessment to evaluate participants' conceptual and code-level understanding of the Pancake Sort algorithm.

5.1.3 Stage 3: Survey and Interview (10 min). All participants completed a post-study questionnaire including UMUX-Lite [25] and a short-form version of NASA-TLX [15], with items adapted to the study context. The session concluded with a semi-structured interview tailored to the participant's role. For student participants, the interview focused on usability, feature comprehension, and learning experience. For instructor participants, the interview further explored pedagogical value, potential for classroom integration, and comparisons with existing teaching practices.

5.1.4 Data Collection and Analysis. We collected screen recordings (including audio), along with interaction logs and survey responses. We summarized quantitative metrics (UMUX-Lite, NASA-TLX, and feature ratings) across all 20 participants to assess overall system usability. We analyzed qualitative data from think-aloud sessions and interviews using a lightweight coding approach. The first author reviewed all recordings and transcripts and conducted an initial round of coding to identify recurring patterns. We then summarized the frequency of these codes and selected representative excerpts to illustrate common challenges in trace organization, abstraction, and interaction during algorithm construction.

6 Results

6.1 Challenges in Pen-and-Paper Sketching

By analyzing handwriting data from two pen-and-paper sessions, alongside think-aloud protocols and follow-up interviews, we identified several critical challenges that participants faced during manual sketching. Figure 2 summarizes the recurring issues observed in participants' hand-drawn algorithm sketches across both stages.

Missed drawing errors. As the complexity of manual tracing increased, participants frequently committed inadvertent clerical errors. In Figure 2(a), P01 marked an incorrect comparison result (*8<5*). In P04's manuscript, a comparison line was written as *1,5* when the adjacent pair should be *4,5*. In P15's flip trace, the sequence *214* was incorrectly written as *421* after flipping.

Omission of procedural steps. To mitigate drawing overhead, several participants skipped intermediate algorithmic states. In Figure 2(b), participants used shorthand marks to omit later iterations. P08 used "->" to indicate that subsequent rounds were skipped and stopped the trace early, while E04 used "|" to mark omitted steps.

Difficulty depicting flip actions. Participants reported that representing the *flip* operation in Pancake Sort was particularly challenging. In Figure 2(c), participants adopted different visual strategies to track *flip* transformations. P08 used lines to mark which numbers changed position in each flip, while P14 used geometric shapes to represent changes in different parts of the flipped segment.

Lack of structure. Many sketches produced during step-by-step tracing of concrete values lacked clear structural organization. In Figure 2(d), P07 recognized the presence of a loop but left only "for", without specifying iteration boundaries. P10's sketch consisted largely of unannotated value sequences with minimal structural markers, making the process difficult to reconstruct and verify.

Varying levels of abstraction. In Figure 2(e), participants showed markedly different abstraction styles. E01 used an abstract array representation without concrete values, marking swap symbols across array elements to indicate Bubble Sort. P11 explicitly wrote several rounds of comparisons up to the final swap, while P03 used a single concrete swap instance to represent the resulting behavior. In contrast, P13 preferred a metaphorical representation, using bubbles to depict the largest bubble rising to the top.

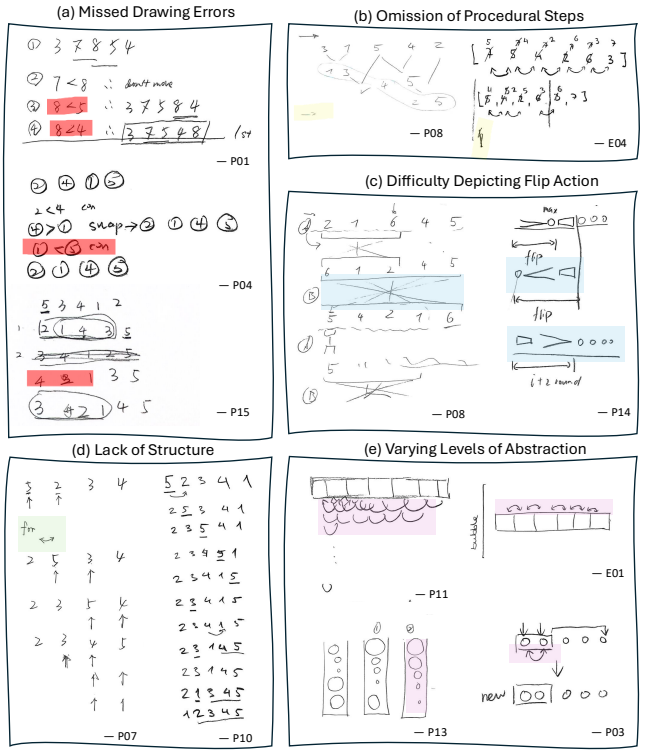


Figure 2: Common issues observed in participants' pen-and-paper algorithm sketches, including missed drawing errors, omission of procedural steps, difficulty representing flip operations, lack of structure, and varying levels of abstraction.

6.2 Workload and Usability Results

As shown in Figure 3, participants reported high scores for *Usefulness* ($M = 6.25, SD = 0.72$) and *Usability* ($M = 5.85, SD = 0.93$) on a 7-point scale. Following the standard UMUX-Lite scoring [25], these item ratings transform to an overall score of $M = 83.75$ ($SD = 11.30$), corresponding to an A grade on the SUS curved grading scale [24]. These results indicate that the system's interface is intuitive and minimizes *extraneous cognitive load* [42], allowing users to focus on the algorithmic tasks rather than navigating the tool itself. Notably, the NASA-TLX results revealed moderate to high *Mental Demand* ($M = 4.8, Med = 5$) and *Effort* ($M = 4.6, Med = 5$). Following the principles of *desirable difficulties* [2], we interpret this not as a sign

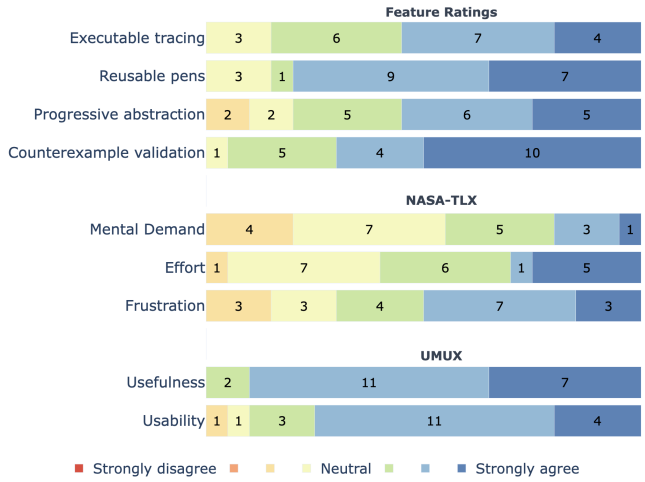


Figure 3: Participant ratings of system feature usefulness, workload, and overall experience, measured on a 7-point Likert scale (1 = strongly disagree, 7 = strongly agree).

of system frustration, but as a reflection of the *intrinsic load* (processing complex sorting logic) and *germane load* (active schema construction and generalization) inherent in algorithmic reasoning. This alignment suggests that while SketchMind lowers the barrier to entry through high usability, it preserves the cognitive engagement necessary for deep learning, effectively bridging the gap between concrete tracing and abstract algorithm construction.

6.3 Perceptions of Core System Features

6.3.1 Executable tracing. Executable tracing was rated in the agreement range ($M = 5.60$, range = 4–7). Participants commonly linked it to easier step tracking and less mental simulation.

Perceived reduction in manual tracing effort. Executable tracing reduces the need for manual state simulation and makes step tracking less effortful. Participants noted that the system "*saves you the process of calculating the loop*" and "*does it for you*" (P14, P8), making interaction "*very automatic*" and "*very direct*" (P12, P10).

Operation animation improves understanding. Participants reported that "*without animation I could not understand this algorithm*" (P7), as element movement made operations such as swapping visible, and helped "*form intuition*" by making processes "*more intuitively and clearly*" observable than textual descriptions (P14). P9 also described relying less on mental simulation, noting no longer having to "*figure out how to flip the array in my mind.*"

Supporting step-by-step inspection and strategy reflection. Executable tracing supported stepwise reasoning by making intermediate states explicit and persistent. Participants valued the ability to "*record the results of each step*" (P12), "*check back the blocks*" to locate errors (P3), and "*restore your whole process*" for reflection (P10). Coordinated views of operations and results further grounded reasoning: users could see "*what it was swapping*" (P11) while also "*check each step*" (P13). This traceability supported debugging, enabling participants to follow the process step by step and determine "*what to compare next*" (P6, P7), and further facilitated iterative strategy refinement.

6.3.2 Reusable pens. Reusable pens received high ratings ($M = 6.00$, range = 4–7). Participants described them as a practical way to reuse action sequences and reduce repeated drawing.

From actions to function-like abstractions and code-like thinking. Participants understood reusable pens as a way to encapsulate action sequences into higher-level, function-like structures. Several explicitly described them in programming terms: P4 and P7 referred to a pen as "*a function*" that can be "*used directly*" or "*defined*". Building on this, participants also related these structures to familiar code logic. P13 observed that merging produces behavior similar to "*If-Else*", and P12 described the resulting abstraction as a "*function*" forming "*a cycle*". P10 further noted that "*[the] function name and process ... help [learners] to write the code*".

Perceived reduction in repetition. Participants perceived reusable pens as reducing repeated drawing and the effort involved in reconstructing recurring action sequences. P8 contrasted pens with paper, where repeated redrawing felt "*quite annoying*", noting that with a reusable pen they could simply "*take it and just put it in there*". P13 and P14 likewise noted that merging reduces effort by avoiding having to "*draw two steps with two brushes every time*" and "*saves you the process of calculating the loop*".

6.3.3 Multi-level trace views. Progressive abstraction showed greater dispersion ($M = 5.50$, range = 3–7). Comments suggest that some participants used the value, index, and parameter views to reason beyond specific values, while others needed more time to adapt to these representations.

From concrete values to abstract representations. P4 described the views as "*a three dimensional picture*" consisting of value, index, and generalized parameter patterns. P10 notes that value is most intuitive while parameter view is "*more like coding*". Participants also switched between views to turn concrete execution steps into reusable logic. In the Pancake Sort task, for example, P5 completed one placement step in the value view by finding the maximum element, flipping it to the front, and then flipping it to the end. P5 then moved to the index view and combined these operations into a reusable procedure for moving the maximum value in a range to its final position, explaining that the index and parameter views helped express "*generalizable logic*". P8 summarized this progression as transforming an "*instantiated exercise into an abstract learning experience*".

Reasoning about traces across instances. Participants used the abstraction views to discuss how the same trace might apply across multiple instances. P13 noted that "*abstraction with indices helps generalization*", while P4 explained that index and parameter views enable application to larger inputs and facilitate movement toward a "*generalized function*". P5 similarly recognized that the approach extends "*to other examples as well*", and P9 valued seeing how simple cases generalize through parameters. P11's inquiry about adapting the number of passes "*to any length*" further indicated reflection on how an instance-specific reasoning might extend to general algorithm design.

6.3.4 Counterexample-based validation. Counterexample-based validation had the highest mean rating ($M = 6.15$, range = 4–7). Participants found it surfaced errors that did not appear in initial examples and supported revision of earlier steps.

Reveals hidden logical errors. Participants reported that counterexamples expose mistakes that are not visible in initial examples and was one of the system's most distinctive strengths. P11 noted that even when the current test seemed fine, the system still produced a warning showing the solution "*wasn't perfect*". For instance, P11's Bubble Sort trace appeared to work on ([3,1,4,2]), but failed on ([3,2,4,1]) because it omitted the final comparison. P12 called the counterexample "*very important*" because otherwise they might have wrongly believed that "*all two rounds would be fine*". P7 likewise valued that the tool directly "*gives the wrong example so you can locate where logic breaks*".

Encourage debugging and reflection. Counterexamples did not merely signal failure; they prompted deeper reflection. After seeing the failure on ([3,2,4,1]), P11 revisited the trace, identified the missing final comparison, and added the omitted step. P9 said the feedback "*encouraged me to look deeply into the code*" on the right rather than focus only on the sketch. P5 felt that the system "*encourages you to think in the opposite direction*" and makes it easier to reflect where "*the whole logic failed*". P4, P6, and P7 all described counterexamples as making it easier to inspect the previous process and identify a missing or faulty step. P10 similarly interpreted a wrong example as evidence that "*something is wrong in the previous process*" or perhaps in earlier pen construction. In this way, validation functioned as an immediate trigger for logic revision.

6.4 Instructors Interview Results

6.4.1 Role in Teaching Contexts. Teaching-staff feedback revealed varying perspectives on how to integrate the tool into instruction, particularly regarding instructional sequencing and the structure of specific learning activities. E4 framed it primarily as a verification practice medium, arguing learners should arrive with prior conceptual grounding; translated into course design, this implies a *lecture-first, verification-second* sequence in which instructors first establish algorithm invariants and canonical procedures, then use the tool for checking, replay, and error localization. E3 emphasized stronger in-class interactivity, noting that live use can be highly participatory (e.g., "*call kids up*" to draw together and inspect outcomes in real time). E1 and E2 placed relatively more weight on self-paced/homework deployment: they argued learners gain more by *doing* than passively watching, so class time is better used for brief conceptual setup plus demonstration, with deeper exploratory construction assigned after class at each learner's own pace. E3 also added a practical condition for independent use: without additional prompted tasks or challenge variations, many learners may stop after one successful completion. A practical hybrid schedule emerging from these views is: short in-class concept briefing and one guided demo, followed by structured homework tasks with progressive prompts (basic trace, counterexample repair, and abstraction/merge), then next-class debrief on common failures.

6.4.2 Support Across Learning Stages. Instructors perceived that the system is particularly suitable for novice learners. E5 characterized it as "*very good for introductory learning,*" while E1 and E3 similarly reported high overall learnability. Instructors attributed the system's perceived value partly to its active, interaction-driven format. Participants emphasized that "*doing it yourself helps more than watching*" (E4), and that dynamic visualization—such as "*seeing numbers move*" (E2)—makes algorithmic processes more concrete and intuitive than static representations. Several instructors contrasted this with pen-and-paper approaches, noting that animation enables learners to "*see the inner working of the algorithm*" (E3) and better grasp large-scale transformations. In addition, composing operations into reusable units was seen as valuable for revealing repetition and structure, helping learners move from isolated steps toward understanding loops and general patterns. E2 said the feature shifts learners from thinking about "*this array*" to "*arrays in general*", and E1 emphasized that it helps users realize they need a general solution rather than an example-specific sketch.

6.4.3 Transfer and Reference Outcomes. Instructors perceived the system as offering a possible pedagogical bridge from visual manipulation to more formalized procedures. E1 and E3 emphasized that composing pens makes repetition and generalization visible in ways "*static paper rarely does.*" E5 noted gains in "*visualization ability,*" helping novices transform concrete manipulations into algorithm schemas that can later "*transfer to pseudocode or code*" (E2). As a descriptive reference outcome, all participants correctly answered the conceptual and code-related post-task questions on Pancake Sort; details of these questions are provided in the supplementary study materials.

7 Discussion

7.1 Transition from Examples to Abstractions

A central challenge in programming education is bridging the gap between concrete examples and generalized procedures [37, 38]. SketchMind scaffolds this transition through progressive abstraction: learners begin with value-specific traces and incrementally lift that logic to indices and symbolic parameters. This hierarchy parallels findings in subgoal-oriented learning, where breaking complex tasks into manageable functional units facilitates transfer [5, 30, 32]. By allowing abstraction to emerge from interaction rather than requiring it upfront, the system supports the internal reorganization of mental models into durable understanding [7, 47]. This process also suggests a viable pathway toward formal implementation. SketchMind functions as an intermediate representational layer—a bridge between perceptual interaction and formal coding. As participants suggested, integrating this workflow with platforms like "*LeetCode*" could facilitate a staged pipeline: learners first validate algorithmic logic through sketches before tackling the syntactic overhead of conventional IDEs. This separation of planning from implementation is well-supported by prior research on novice programming workflows [29, 33, 40].

7.2 Learning through Counterexamples

The counterexample mechanism provides concrete evidence when a learner's current logic does not generalize, reinforcing the pedagogical principle that feedback is most effective when it exposes the gap between current and target understanding [16]. Rather than

simply indicating that an algorithm is incorrect, minimal counterexamples reveal where the logic breaks down while preserving the learner's interaction context. This interaction also connects to computational thinking practices beyond any particular sorting algorithm. Weintrop et al. describe practices such as testing solutions, debugging, decomposition, and recognizing patterns across computational processes [49]. In SketchMind, replaying a trace on a counterexample allows learners to test an informal hypothesis about an algorithm and compare its expected and observed behavior. This design manages *intrinsic cognitive load* by segmenting the debugging process into targeted reasoning episodes [22, 41, 46]. Furthermore, our findings suggest that counterexamples offer a flexible foundation for balancing open-ended exploration with appropriately timed guidance—a critical tension in the learning sciences [16, 19]. By prioritizing global validation, SketchMind preserves the "productive struggle" necessary for self-correction and deep reflection on algorithmic correctness. However, mathematical minimality does not necessarily align with pedagogical clarity: the closest failing input may not always make the underlying misconception most apparent. Future systems could therefore consider pedagogical criteria alongside input distance when selecting counterexamples. As noted by instructors, this mechanism could be further enriched with localized feedback cues (e.g., highlighting a specific operation group). Such a multi-level strategy would allow learners to navigate errors more efficiently without prematurely revealing the solution, thereby maintaining the reflective value of the debugging process.

7.3 Towards General Purpose SketchMind

The current SketchMind prototype is a proof of concept for array-based data structures and predominantly linear execution traces, demonstrated through Bubble Sort and Pancake Sort. These two algorithms provide a controlled progression within the same visual domain. Bubble Sort demonstrates the basic trace-to-pen workflow, whereas Pancake Sort serves as a near-transfer task that preserves the array representation while introducing range-based reasoning through *find-max* and *prefix-flip* operations. We distinguish between the system infrastructure that can be reused across algorithms and the algorithm-specific components that must be extended. The reusable infrastructure includes the sketch canvas, execution-trace representation, replay mechanism, pen-composition workflow, and counterexample-search pipeline. While primitive pen semantics, visual data models, abstraction views, and SMT predicates are specific to the supported algorithmic domain. For example, extending SketchMind from Bubble Sort to Pancake Sort reused the existing trace, replay, composition, and validation infrastructure, but required new support for range selection, *find-max* and *flip* semantics, predicates for range-based reasoning, and composition logic linking a selected range to its corresponding operation.

SketchMind is not currently a general-purpose end-user authoring environment for arbitrary algorithms. Users can compose existing primitive pens into reusable pens, but they cannot define the semantics of new primitive pens through the interface. Supporting a new class of operations requires developer-authored visual representations, operation semantics, abstraction views, and SMT predicates. Extending SketchMind to a substantially different domain would require more extensive additions. For example, supporting Dijkstra's algorithm would require a graph-based visual state, representations of distance labels and visited status, and new primitives such as *RelaxEdge*, *MarkVisited*, and *ExtractMin*. Counterexample generation would likewise require new domain-specific predicates encoding the validity and effects of these operations. This extension also raises a scaling challenge for the pen set. As primitive and reusable pens accumulate, their discoverability and visual organization may become difficult, resembling the palette-management challenges of block-based programming environments. Possible design directions include scoped palettes that expose only contextually relevant pens or phase-based grouping that organizes pens according to stages of an algorithm.

Another important direction is to better interpret the rich visual language that users already employ when sketching. In our study, participants naturally used layout, grouping, and variations in size to express meaning. For example, in Figure 2, one participant drew actual "bubbles" and used the size of each bubble to represent the value, while others used different visual encodings such as numbers, bars, or circles. These variations suggest that learners bring rich and expressive representations when reasoning about the same algorithm. Future versions of the system could better leverage these cues, allowing it to infer higher-level structure without requiring precise or standardized input (e.g., [51]).

7.4 Limitations

SketchMind represents an early exploration of executable sketching for algorithm construction. The current implementation supports two array-based sorting algorithms with local, sequential operations. Extending SketchMind to recursive, graph, dynamic-programming, or multi-data-structure algorithms would require representations for recursive calls, branching execution, intermediate states, and interacting data structures. Our evaluation focuses on usability rather than learning outcomes such as transfer, retention, or performance on unseen algorithms. Future work should examine these broader algorithmic structures and evaluate the approach through comparative and longitudinal studies.

8 Conclusion

In this paper, we presented SketchMind, a proof-of-concept sketch-based system that transforms tracing into an interactive process for algorithm construction through executable tracing, reusable pens, multi-level trace views, and counterexample-based validation. In our preliminary user study, participants perceived less effort in manually tracking execution states, used reusable pens and multiple trace views to organize and abstract their traces, and used counterexamples to reflect on whether their logic generalized beyond the original example.

Acknowledgments

This work was supported by the Swiss National Science Foundation (SNSF) under grant CR00-5-239839 and by the Joint Doctoral Program in the Learning Sciences at ETH Zurich and EPFL. We thank Dr. Lahari Goswami, our labmates, and all study participants for their valuable feedback, support, and contributions to this work.

References

[1] Dave Berque, Terri Bonebright, and Michael Whitesell. 2004. Using pen-based computers across the computer science curriculum. In *Proceedings of the 35th SIGCSE Technical Symposium on Computer Science Education* (Norfolk, Virginia, USA) *(SIGCSE '04)*. Association for Computing Machinery, New York, NY, USA, 61–65. https://doi.org/10.1145/971300.971324

[2] Robert Bjork. 1994. *Memory and Meta-memory Considerations in the Training of Human Beings*. 185–205. https://doi.org/10.7551/mitpress/4561.003.0011

[3] Marc H. Brown and Robert Sedgewick. 1984. A system for algorithm animation. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '84)*. Association for Computing Machinery, New York, NY, USA, 177–186. https://doi.org/10.1145/800031.808596

[4] Sarah Buchanan, Brandon Ochs, and Joseph J. LaViola Jr. 2012. CSTutor: a pen-based tutor for data structure visualization. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education* (Raleigh, North Carolina, USA) *(SIGCSE '12)*. Association for Computing Machinery, New York, NY, USA, 565–570. https://doi.org/10.1145/2157136.2157297

[5] Richard Catrambone. 1998. The subgoal learning model: Creating better examples so that students can solve novel problems. *Journal of Experimental Psychology: General* 127 (1998), 355–376. https://api.semanticscholar.org/CorpusID:3817537

[6] Weihao Chen, Xiaoyu Liu, Jiacheng Zhang, Ian Iong Lam, Zhicheng Huang, Rui Dong, Xinyu Wang, and Tianyi Zhang. 2023. MIWA: Mixed-Initiative Web Automation for Better User Control and Confidence. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) *(UIST '23)*. Association for Computing Machinery, New York, NY, USA, Article 75, 15 pages. https://doi.org/10.1145/3586183.3606720

[7] National Research Council, Board on Behavioral, Sensory Sciences, Committee on Developments in the Science of Learning with additional material from the Committee on Learning Research, and Educational Practice. 2000. *How people learn: Brain, mind, experience, and school: Expanded edition*. Vol. 1. National Academies Press.

[8] Kathryn Cunningham. 2017. The Effect of Sketching and Tracing on Instructors' Understanding of Student Misconceptions. In *Proceedings of the 2017 ACM Conference on International Computing Education Research* (Tacoma, Washington, USA) *(ICER '17)*. Association for Computing Machinery, New York, NY, USA, 301–302. https://doi.org/10.1145/3105726.3105746

[9] Kathryn Cunningham, Sarah Blanchard, Barbara Ericson, and Mark Guzdial. 2017. Using Tracing and Sketching to Solve Programming Problems: Replicating and Extending an Analysis of What Students Draw. In *Proceedings of the 2017 ACM Conference on International Computing Education Research* (Tacoma, Washington, USA) *(ICER '17)*. Association for Computing Machinery, New York, NY, USA, 164–172. https://doi.org/10.1145/3105726.3106190

[10] Allen Cypher, Daniel C. Halbert, David Kurlander, Henry Lieberman, David Maulsby, Brad A. Myers, and Alan Turransky (Eds.). 1993. *Watch What I Do: Programming by Demonstration*. MIT Press, Cambridge, MA, USA.

[11] Scott Grissom, Myles F. McNally, and Tom Naps. 2003. Algorithm visualization in CS education: comparing levels of student engagement. In *Proceedings of the 2003 ACM Symposium on Software Visualization* (San Diego, California) *(SoftVis '03)*. Association for Computing Machinery, New York, NY, USA, 87–94. https://doi.org/10.1145/774833.774846

[12] Aditya Gunturu, Yi Wen, Nandi Zhang, Jarin Thundathil, Rubaiat Habib Kazi, and Ryo Suzuki. 2024. Augmented Physics: Creating Interactive and Embedded Physics Simulations from Static Textbook Diagrams. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) *(UIST '24)*. Association for Computing Machinery, New York, NY, USA, Article 144, 12 pages. https://doi.org/10.1145/3654777.3676392

[13] Philip J. Guo. 2013. Online python tutor: embeddable web-based program visualization for cs education. In *Proceeding of the 44th ACM Technical Symposium on Computer Science Education* (Denver, Colorado, USA) *(SIGCSE '13)*. Association for Computing Machinery, New York, NY, USA, 579–584. https://doi.org/10.1145/2445196.2445368

[14] Philip J. Guo, Sean Kandel, Joseph M. Hellerstein, and Jeffrey Heer. 2011. Proactive wrangling: mixed-initiative end-user programming of data transformation scripts. In *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology* (Santa Barbara, California, USA) *(UIST '11)*. Association for Computing Machinery, New York, NY, USA, 65–74. https://doi.org/10.1145/2047196.2047205

[15] Sandra Hart. 2006. Nasa-Task Load Index (NASA-TLX); 20 Years Later. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* 50 (10 2006), 904–908. https://doi.org/10.1177/154193120605000909

[16] John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77 (03 2007), 81–112. https://doi.org/10.3102/003465430298487

[17] Christopher Hundhausen, Sarah Douglas, and John Stasko. 2002. A Meta-Study of Algorithm Visualization Effectiveness. *Journal of Visual Languages & Computing* 13 (06 2002), 259–290. https://doi.org/10.1006/jvlc.2002.0237

[18] Md Athar Imtiaz, Andrew Luxton-Reilly, and Beryl Plimmer. 2018. ThinkInk - An Intelligent Sketch Tool for Learning Data Structures. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) *(CHI EA '18)*. Association for Computing Machinery, New York, NY, USA, 1–6. https://doi.org/10.1145/3170427.3188441

[19] Paul Kirschner, John Sweller, and Richard Clark. 2006. Why Minimal Guidance During Instruction Does Not Work: An Analysis of the Failure of Constructivist, Discovery, Problem-Based, Experiential, and Inquiry-Based Teaching. *Educational Psychologist* 41 (06 2006). https://doi.org/10.1207/s15326985ep4102_1

[20] Tessa Lau. 2009. Why Programming by Demonstration Systems Fail: Lessons Learned for Usable AI. *AI Mag.* 30, 4 (Dec. 2009), 65–67. https://doi.org/10.1609/aimag.v30i4.2262

[21] Tessa A. Lau and Daniel S. Weld. 1998. Programming by demonstration: an inductive learning formulation. In *Proceedings of the 4th International Conference on Intelligent User Interfaces* (Los Angeles, California, USA) *(IUI '99)*. Association for Computing Machinery, New York, NY, USA, 145–152. https://doi.org/10.1145/291080.291104

[22] Jimmie Leppink, Fred Paas, Cees Van der Vleuten, Tamara Gog, and Jeroen J. G. Van Merrienboer. 2013. Development of an instrument for measuring different types of cognitive load. *Behavior Research Methods* 45 (04 2013), 1058–1072. https://doi.org/10.3758/s13428-013-0334-1

[23] Sorin Lerner. 2020. Projection Boxes: On-the-fly Reconfigurable Visualization for Live Programming. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–7. https://doi.org/10.1145/3313831.3376494

[24] James Lewis. 2018. Measuring Perceived Usability: The CSUQ, SUS, and UMUX. *International Journal of Human-Computer Interaction* 34 (01 2018), 1–9. https://doi.org/10.1080/10447318.2017.1418805

[25] James R. Lewis, Brian S. Utesch, and Deborah E. Maher. 2013. UMUX-LITE: when there's no time for the SUS *(CHI '13)*. Association for Computing Machinery, New York, NY, USA, 2099–2102. https://doi.org/10.1145/2470654.2481287

[26] Chuanjun Li, Timothy S. Miller, Robert C. Zeleznik, and Joseph J. LaViola. 2008. AlgoSketch: algorithm sketching and interactive computation. In *Proceedings of the Fifth Eurographics Conference on Sketch-Based Interfaces and Modeling* (Annecy, France) *(SBM'08)*. Eurographics Association, Goslar, DEU, 175–182.

[27] Raymond Lister, Elizabeth S. Adams, Sue Fitzgerald, William Fone, John Hamer, Morten Lindholm, Robert McCartney, Jan Erik Moström, Kate Sanders, Otto Seppälä, Beth Simon, and Lynda Thomas. 2004. A multi-national study of reading and tracing skills in novice programmers. In *Working Group Reports from ITiCSE on Innovation and Technology in Computer Science Education* (Leeds, United Kingdom) *(ITiCSE-WGR '04)*. Association for Computing Machinery, New York, NY, USA, 119–150. https://doi.org/10.1145/1044550.1041673

[28] Raymond Lister, Beth Simon, Errol Thompson, Jacqueline L. Whalley, and Christine Prasad. 2006. Not seeing the forest for the trees: novice programmers and the SOLO taxonomy. In *Proceedings of the 11th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education* (Bologna, Italy) *(ITICSE '06)*. Association for Computing Machinery, New York, NY, USA, 118–122. https://doi.org/10.1145/1140124.1140157

[29] Shuai Ma, Junling Wang, Yuanhao Zhang, Xiaojuan Ma, and April Yi Wang. 2025. DBox: Scaffolding Algorithmic Programming Learning through Learner-LLM Co-Decomposition. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 585, 20 pages. https://doi.org/10.1145/3706598.3713748

[30] Lauren E. Margulieux, Mark Guzdial, and Richard Catrambone. 2012. Subgoal-labeled instructional material improves performance and transfer in learning to develop mobile applications. In *Proceedings of the Ninth Annual International Conference on International Computing Education Research* (Auckland, New Zealand) *(ICER '12)*. Association for Computing Machinery, New York, NY, USA, 71–78. https://doi.org/10.1145/2361276.2361291

[31] Mikaël Mayer, Gustavo Soares, Maxim Grechkin, Vu Le, Mark Marron, Oleksandr Polozov, Rishabh Singh, Benjamin Zorn, and Sumit Gulwani. 2015. User Interaction Models for Disambiguation in Programming by Example. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology* (Charlotte, NC, USA) *(UIST '15)*. Association for Computing Machinery, New York, NY, USA, 291–301. https://doi.org/10.1145/2807442.2807459

[32] Briana B. Morrison, Lauren E. Margulieux, and Mark Guzdial. 2015. Subgoals, Context, and Worked Examples in Learning Computing Problem Solving. In *Proceedings of the Eleventh Annual International Conference on International Computing Education Research* (Omaha, Nebraska, USA) *(ICER '15)*. Association for Computing Machinery, New York, NY, USA, 21–29. https://doi.org/10.1145/2787622.2787733

[33] Laurie Murphy, Renée McCauley, and Sue Fitzgerald. 2012. 'Explain in plain English' questions: implications for teaching. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education* (Raleigh, North Carolina, USA) *(SIGCSE '12)*. Association for Computing Machinery, New York, NY, USA, 385–390. https://doi.org/10.1145/2157136.2157249

[34] Thomas Naps, Stephen Cooper, Boris Koldehofe, Charles Leska, Guido Rößling, Wanda Dann, Ari Korhonen, Lauri Malmi, Jarmo Rantakokko, Rockford J. Ross, Jay Anderson, Rudolf Fleischer, Marja Kuittinen, and Myles McNally. 2003. Evaluating the educational impact of visualization. In *Working Group Reports from*

ITiCSE on Innovation and Technology in Computer Science Education (Thessaloniki, Greece) *(ITiCSE-WGR '03)*. Association for Computing Machinery, New York, NY, USA, 124–136. https://doi.org/10.1145/960875.960540

[35] Thomas L. Naps, Guido Rößling, Vicki Almstrum, Wanda Dann, Rudolf Fleischer, Chris Hundhausen, Ari Korhonen, Lauri Malmi, Myles McNally, Susan Rodger, and J. Ángel Velázquez-Iturbide. 2002. Exploring the role of visualization and engagement in computer science education. *SIGCSE Bull.* 35, 2 (June 2002), 131–152. https://doi.org/10.1145/782941.782998

[36] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. 2009. Scratch: programming for all. *Commun. ACM* 52, 11 (Nov. 2009), 60–67. https://doi.org/10.1145/1592761.1592779

[37] Anthony Robins, Janet Rountree, and Nathan Rountree. 2003. Learning and Teaching Programming: A Review and Discussion. *Computer Science Education* 13, 2 (2003), 137–172. https://doi.org/10.1076/csed.13.2.137.14200

[38] Anthony V Robins. 2019. 12 novice programmers and introductory programming. *The Cambridge handbook of computing education research* (2019), 327.

[39] Nazmus Saquib, Rubaiat Habib Kazi, Li-yi Wei, Gloria Mark, and Deb Roy. 2021. Constructing Embodied Algebra by Sketching. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 428, 16 pages. https://doi.org/10.1145/3411764.3445460

[40] IV Smith, David H. and Craig Zilles. 2024. Code Generation Based Grading: Evaluating an Auto-grading Mechanism for "Explain-in-Plain-English" Questions. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1* (Milan, Italy) *(ITiCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 171–177. https://doi.org/10.1145/3649217.3653582

[41] Kenneth W Spence and Janet Taylor Spence. 1967. *Psychology of learning and motivation*. Vol. 1. Academic Press.

[42] John Sweller. 1988. Cognitive load during problem solving: Effects on learning. *Cognitive science* 12, 2 (1988), 257–285.

[43] Steven L. Tanimoto. 2013. A perspective on the evolution of live programming. In *Proceedings of the 1st International Workshop on Live Programming* (San Francisco, California) *(LIVE '13)*. IEEE Press, 31–34.

[44] Blase Ur, Melwyn Pak Yong Ho, Stephen Brawner, Jiyun Lee, Sarah Mennicken, Noah Picard, Diane Schulze, and Michael L. Littman. 2016. Trigger-Action Programming in the Wild: An Analysis of 200,000 IFTTT Recipes. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems* (San Jose, California, USA) *(CHI '16)*. Association for Computing Machinery, New York, NY, USA, 3227–3231. https://doi.org/10.1145/2858036.2858556

[45] Remko van der Lugt. 2005. How sketching can affect the idea generation process in design group meetings. *Design Studies* 26, 2 (2005), 101–122. https://doi.org/10.1016/j.destud.2004.08.003

[46] Jeroen J. G. Van Merrienboer and John Sweller. 2005. Cognitive Load Theory and Complex Learning: Recent Developments and Future Directions. *Educational Psychology Review* 17 (06 2005), 147–177. https://doi.org/10.1007/s10648-005-3951-0

[47] Kurt Vanlehn, Randolph Jones, and Michelene Chi. 1992. A Model of the Self-Explanation Effect. *Journal of the Learning Sciences* 2 (01 1992), 1–59. https://doi.org/10.1207/s15327809jls0201_1

[48] Bret Victor. 2011. Up and Down the Ladder of Abstraction: A Systematic Approach to Interactive Visualization. http://worrydream.com/LadderOfAbstraction/. Accessed: 2026-07-22.

[49] David Weintrop, Elham Beheshti, Michael Horn, Kai Orton, Kemi Jona, Laura Trouille, and Uri Wilensky. 2016. Defining Computational Thinking for Mathematics and Science Classrooms. *Journal of Science Education and Technology* 25 (02 2016). https://doi.org/10.1007/s10956-015-9581-5

[50] David Wood and Gail Ross. 2006. The Role of Tutoring in Problem Solving. *Journal of Child Psychology and Psychiatry* 17 (12 2006), 89 – 100. https://doi.org/10.1111/j.1469-7610.1976.tb00381.x

[51] Haijun Xia, Nathalie Henry Riche, Fanny Chevalier, Bruno De Araujo, and Daniel Wigdor. 2018. DataInk: Direct and Creative Data-Oriented Drawing. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) *(CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3173574.3173797

[52] Ryan Yen, Jian Zhao, and Daniel Vogel. 2025. Code Shaping: Iterative Code Editing with Free-form AI-Interpreted Sketching. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 872, 17 pages. https://doi.org/10.1145/3706598.3713822

A Participant Demographics

The instructor group included one secondary-school CS teacher with three years of teaching experience, one extracurricular coding instructor with five years of youth programming experience, and three university teaching assistants with one to four semesters of experience teaching Algorithms and Data Structures, Algorithms and Probability, and Computer Networks.

Table 3: Demographic profiles of participants (N = 20). 'P' denotes student learners and 'E' instructors; 'Yr' indicates year of study. Among instructors: one high school CS teacher, one extracurricular coding instructor, and three TAs.

ID	G	Age	Field of Study	Degree	Yr
P01	F	25	Electrical Engineering	Bachelor	2
P02	M	24	Artificial Intelligence	Master	2
P03	F	25	Informatics	Master	3
P04	F	25	Linguistics	Master	2
P05	F	23	Computer Science	Bachelor	3
P06	F	25	Geomatics	Master	3
P07	F	26	Vehicle Planning and Control	PhD	3
P08	M	21	Computer Science	Master	1
P09	M	25	Computer Science	Master	2
P10	F	25	Data Science	Master	3
P11	M	19	Computer Science	Bachelor	1
P12	F	21	Mathematics	Bachelor	2
P13	F	25	Computer Science	Master	1
P14	M	24	Robotics, Systems and Control	Master	1
P15	F	28	Computational Linguistics	Master	3
E01	F	20	Computer Science	Bachelor	3
E02	F	25	Computer Science	Master	3
E03	F	24	Computer Science	Master	2
E04	M	21	Computer Science	Bachelor	2
E05	F	20	Computer Science	Bachelor	3

B System Pipeline

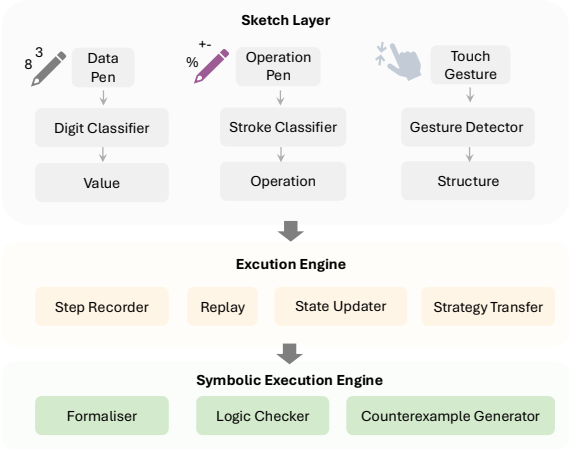


Figure 4: Pipeline of SketchMind. Pen and touch inputs are classified and bound to array elements, operations, and structure. The execution engine records and replays steps across instances. The symbolic execution engine verifies strategies and injects minimal counterexamples.